LAB3: FPGA 응용 회로

실험 후 보고서

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/10/9

대상: LED PWM/RGB PWM/피에조/스텝 모터/MMSS 시계/문자 LCD/UART Echo

검증 환경: VS Code VaporView, Vivado 2026.1, FPGA Spartan

보고서 및 실험의 범위

LED PWM, RGB PWM, 피에조, 스텝 모터, MMSS 시계, 문자 LCD, UART Echo의 총 7개 실험을 대상으로 Vivado 시뮬레이션 및 FPGA 보드 위 검증을 수행하였다. 파형의 관점에서 본 설계는 실험 전 보고서에서 작성하였기 때문에 이 보고서에서는 FPGA 보드 위의 설계를 위주로 설명하겠다.

소스 식별

프로젝트: lab3

기준 커밋: 6b3b06b434656f3209f5eb2e263d71f399239235

실행본: src/, sim/, constraints/

GitHub 주소: https://github.com/Jeong719/ECE2_Lab3.git

제출 태그: v1.0

1. 목적과 공통 실험 환경

실험 목적

조합논리회로와 순차논리회로에 대해서 배운 점을 기반으로 총 7종의 FPGA 응용 회로를 Verilog HDL로 설계하고, 시뮬레이션한 결과를 바탕으로 이를 vivado를 통해 fpga 보드에 구현하여 직접 관찰한다.

공통 실험 환경

사전 검증은 VS Code에서 Icarus Verilog를 이용하여 수행하고, 생성된 VCD 파일을 VaporView로 열어 파형을 분석하였으며, 실험 수행은 Vivado 2026.1 버전을 사용하였으며 fpga보드의 spartan(xc7s75fgga484-1)를 사용하여 스위치, 버튼 및 클록 주파수 조작을 통한 LED 및 세그먼트의 변화를 관측하였다.

검증 범위

시뮬레이션에서는 각 회로의 리셋, 값 유지, 데이터 갱신 및 상태 전이 등 주요 동작을 검증하였다. 실제 보드에서는 버튼과 스위치를 조작하여 대표적인 입력 조건에서 LED와 세그먼트 표시가 설계 의도와 일치하는지 확인하였다.

2. 실험1: LED PWM

설계와 예상 동작 요약

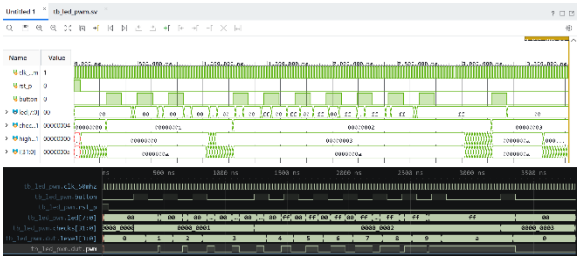
본 회로는 PWM 신호의 듀티비를 조절하여 LED의 밝기를 변경하는 회로다. Press가 할당된 버튼을 누르면 LED의 밝기가 점진적으로 증가하며, 점등된 LED의 밝기는 총 10단계가 존재한다. 마지막 단계 이후 press를 한 번 더 누르면 다시 0단계로 돌아가며, rst_p가 할당된 버튼을 누르면 어든 단계에서든 0단계로 돌아간다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_LED_PWM_PASS checks=4
$finish called at time : 3831 ns
```

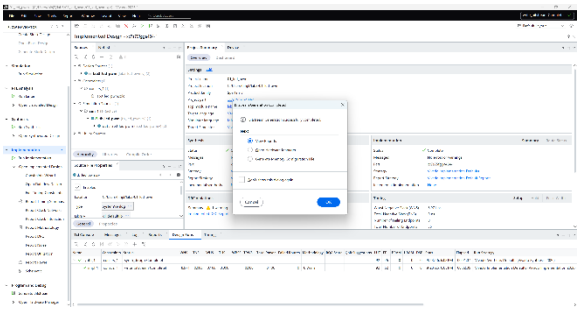


vivado에서 simulation 한 결과 checks=4, 3831ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



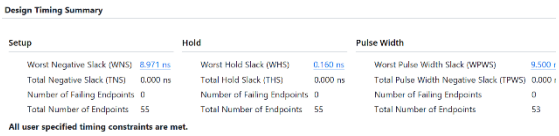
위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



화면 하단을 통해 Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.





위 사진은 업/다운 카운터의 DRC 결과 및 timing 보고서다. 해당 사진처럼 DRC 결과 CFGBVS-1 경고 1건이 표시되었다. 이는 FPGA 구성 뱅크의 전압 관련 속성이 지정되지 않았다는 내용이다.

실제 보드 구현 및 동작 분석

실제 보드에서는 rst_p를 1번 버튼에, press를 2번 버튼에 할당하였다. 1~8번 LED 모두 같은 밝기를 공유하게 구현하였고, 실제로 2번 버튼을 누를 때마다 모든 LED가 밝아지는 것을 확인할 수 있었다. 10단계 밝기에서 2번 버튼을 눌렀을 때 모든 LED가 소등되는 것을 통해 순환하는 구조도 정상적으로 동작하는 것을 확인하였다. 또한 LED 밝기 단계에 상관없이 1번 버튼을 눌렀을 때 모든 LED가 소등되는 것을 확인하였다.

3. 실험2: RGB PWM

설계와 예상 동작 요약

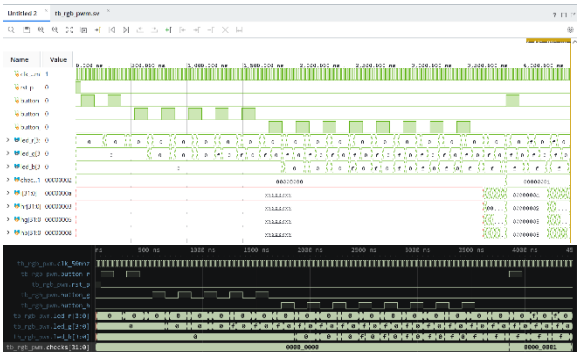
본 회로는 LED_PWM을 활용하여 빨강, 초록, 파랑 LED에 각각 독립적인 PWM 신호를 공급하여 각각의 밝기를 조절하여 혼합색을 조절하는 회로다. Press_r에 해당하는 버튼을 누르면 LED에서 빨간색의 단계가 높아지며, press_g에 해당하는 버튼을 누르면 초록색의 단계가, press_b에 해당하는 버튼을 누르면 파란색의 단계가 높아진다. 실제 LED에 구현되는 색은 세 가지 색의 단계 비율에 따라 다르게 표현된다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_RGB_PWM_PASS checks=2
$finish called at time : 4431 ns
```

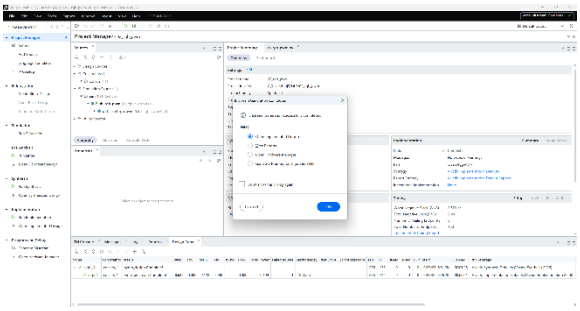


vivado에서 simulation 한 결과 checks=2, 4431ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

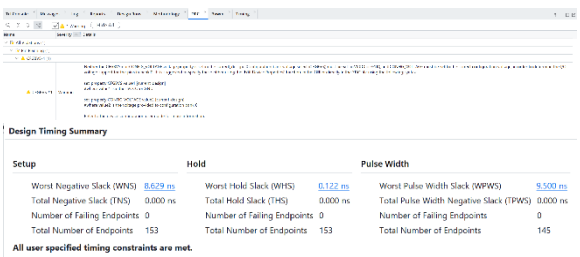


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



화면 하단을 통해 Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 FULL COLOR LED를 통해 혼합색을 확인할 수 있었다. Press_r이 할당된 2번 버튼을 누를수록 LED에 빨간색 불빛이, press_g가 할당된 3번 버튼을 누를수록 LED에 초록색 불빛이, press_b가 할당된 4번 버튼을 누를수록 LED에 파란색 불빛이 밝아지는 것을 확인할 수 있었다. 이후 해당 버튼들을 각각 2,6,8회 순서대로 누르자 빨간색->노란색->매우 밝은 파란색의 순서로 LED가 표시되는 것을 확인하였다.

4. 실험3: 피에조

설계와 예상 동작 요약

본 회로는 일정한 간격으로 출력을 반전시켜 피에조에 사각파를 공급하고 음을 발생시키는 회로다. Rst가 할당된 버튼을 누르지 않으면 보드의 piezo 스피커를 통해 소리가 발생한다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_PIEZO_PASS edges=5
$finish called at time : 551 ns
```



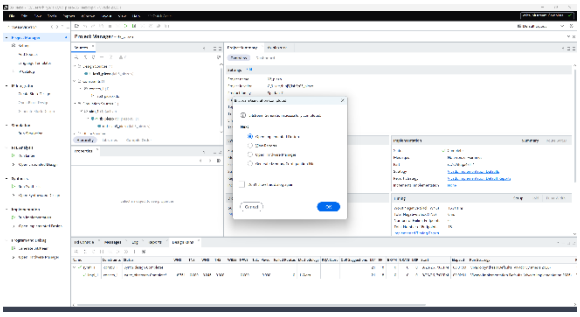
vivado에서 simulation 한 결과 edges=5, 551ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



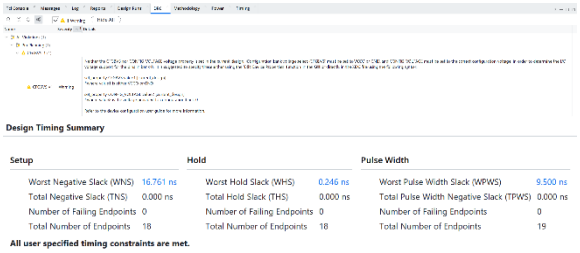


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 Rst를 1번 버튼에 할당하였다. Program device가 완료되자마자 피에조 스피커에서 소리 나는 것을 확인하였다. 다만 1번 버튼을 누르고 있는 동안에는 소리가 나지 않았으며, 보드의 클록 주파수를 변경하자 피에조 스피커에서 출력되는 소리가 낮아지는 것을 확인할 수 있었다.

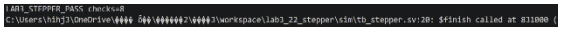
5. 실험4: 스텝 모터

설계와 예상 동작 요약

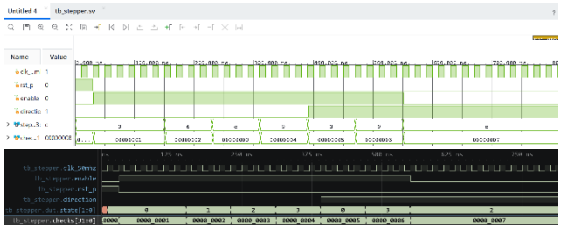
스텝 모터는 전기 신호를 정해진 순서로 주면 일정한 각도씩 회전하는 모터다. State가 바뀔 때마다 스텝 모터는 일정 각도씩 회전하며, state는 enable이 할당된 버튼을 눌렀을 때 변화한다. 이 때, state가 변경되는 순서는 direction이 할당된 버튼을 통해 바꿀 수 있다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_STEPPER_PASS checks=8
$finish called at time : 831 ns
```

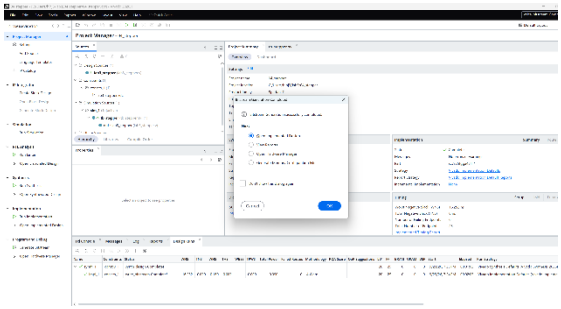


vivado에서 simulation 한 결과 checks=8, 831ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

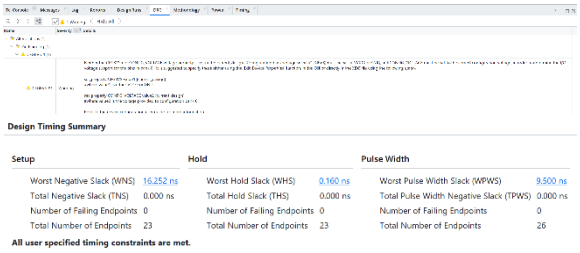


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 2번 버튼에 enable을, 3번 버튼에 direction을 할당하였다. 또한, state는 step motor 아래에 표시되게 구현하였다. 2번 버튼을 짧게 눌렀을 때는 state가 한 단계씩 변하지만, 길게 누르면 state는 지속적으로 변하고 누르는 동안 step motor는 계속 회전하는 것을 확인할 수 있었다. 또한, 3번 버튼을 누르면서 2번 버튼을 누르면 회전 방향을 바꿀 수 있었다.

6. 실험5: MMSS 시계

설계와 예상 동작 요약

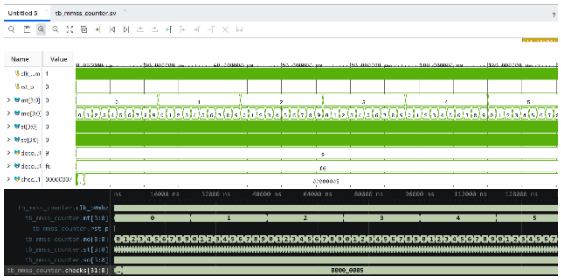
본 회로는 분과 초를 계산하고 7세그먼트에 표시하는 디지털 시계다. Mt, mo, st, so가 할당된 세그먼트를 통해 각각 두 자리의 분과 두 자리의 초를 표시할 수 있다. 1초가 지날 때마다 so에 할당된 세그먼트의 숫자는 1씩 증가하고 so의 숫자가 9에서 0으로 바뀔 때 st의 숫자가 1씩 증가한다. Mo는 st의 숫자가 5에서 0으로 바뀔 때 1씩 증가하며, mt는 mo의 숫자가 9에서 1로 바뀔 때 1씩 증가한다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_MMSS_PASS checks=7
$finish called at time : 144031 ns
```

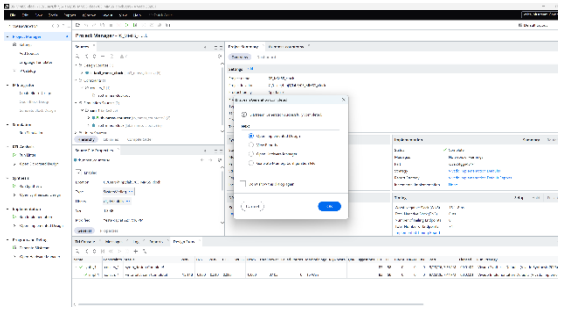


vivado에서 simulation 한 결과 checks=7, 144031ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

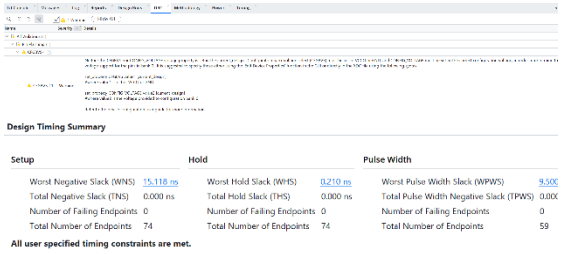


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 실험에서는 8 array segment의 끝 4자리를 통해 시계를 구현하였다. 앞자리부터 mt, mo, st, so를 할당하였으며, 실제 실험 시간 관계상 1분, 즉 mo의 숫자가 0에서 1로 바뀌는 것까지만 확인하였다. So가 9로 된 후 1초가 지나자 so의 숫자가 0이 되는 동시에 st의 숫자가 1씩 증가하는 것을 확인하였고 st의 숫자가 5로 된 후 10초가 지나자 st의 숫자가 0이 되는 동시에 mo의 숫자가 1 증가하는 것을 확인하였다.

7. 실험6: 문자 LCD

설계와 예상 동작 요약

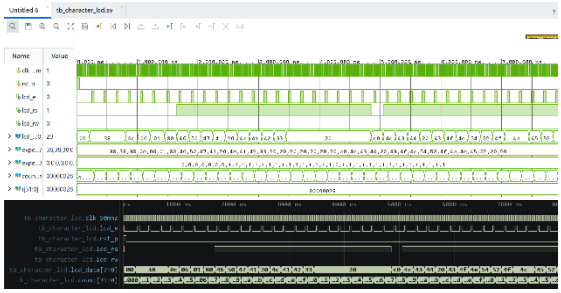
본 회로는 LCD에 명령과 문자 데이터를 정해진 순서와 타이밍으로 전달하여 글자를 표시하는 회로다. 따로 조작에 대한 설계는 존재하지 않고 소스 코드에서 설계된 대로 LCD 패널에 문자를 표시할 수 있다. 이 실험에서는 'LCD CONTROLLER'가 표시되게끔 설계하였다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_LCD_PASS bytes=40
$finish called at time : 8130 ns
```

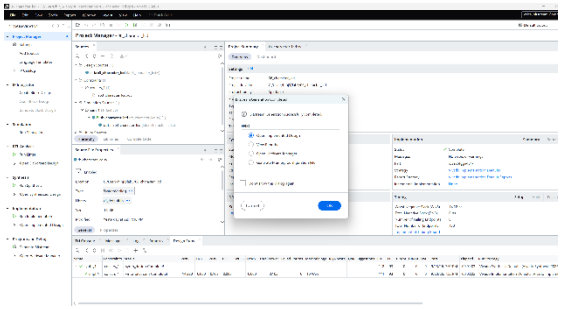


vivado에서 simulation 한 결과 bytes=40, 8130ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

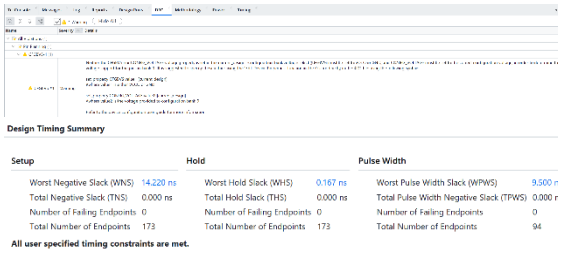


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 비록 두 사진의 신호가 배치된 순서는 다르지만, 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 program device가 완료되는 순간 LCD 패널에 코드에서 설계한 'LCD CONTROLLER'가 표시되는 것을 확인하였다. 이 때, rst가 할당된 1번 버튼을 누르면 잠시나마 LCD 패널이 깜빡이는 것을 통해 리셋 명령이 정상적으로 동작하는 것을 확인할 수 있다.

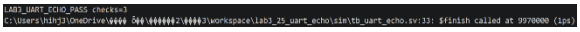
8. 실험7: UART Echo

설계와 예상 동작 요약

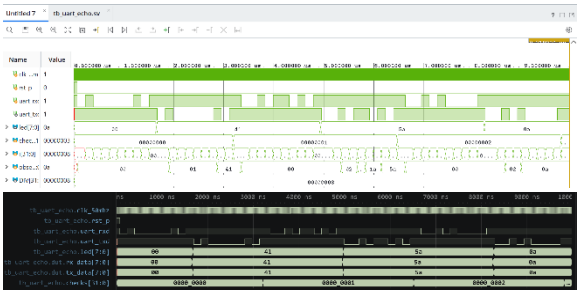
본 회로는 PC에서 UART로 수신한 문자를 같은 값으로 다시 송신하는 에코 회로다. PC에서 특정 문자를 입력하면 해당 문자에 해당하는 LED 배열이 표시되게끔 설계하였다. 다음은 Microchip의 UART 실습 자료에서는 문자 Echo를 이용하여 실시간 통신 동작을 관찰한다고 설명한다.[1]

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB3_UART_ECHO_PASS checks=3
$finish called at time : 9910 ns
```

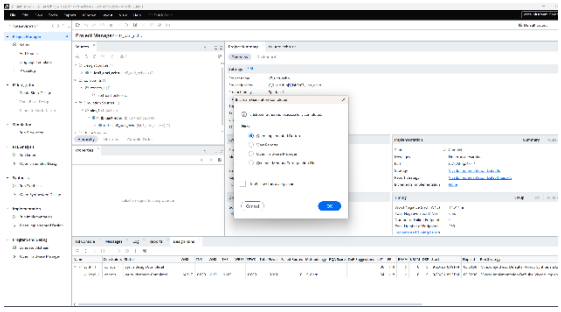


vivado에서 simulation 한 결과 checks=3, 9910ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

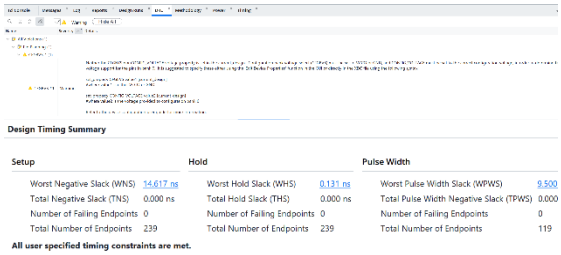


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 Tera Term 및 vscode 터미널을 이용하여 보드에 문자를 송신하였다. PC에서 어떤 문자를 입력하였을 때 해당 문자의 ASCII 코드에 대응하는 값이 2진수에 따라 보드의 LED에 표시되는 것을 확인하였다. 예를 들어, A(0x41)를 입력했을 때는 4에 해당하는 LED2번과 1에 해당하는 LED8번이 점등되는 것을 확인하였다. 이후 Z와 줄바꿈에 대해서도 정상적으로 동작하는 것을 확인하였다.

A 입력



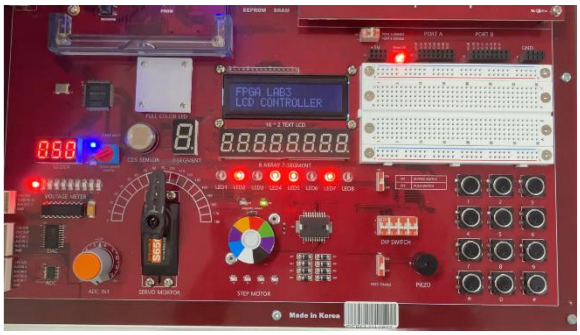
A의 아스키 코드에 해당하는 LED 점등



Z 입력



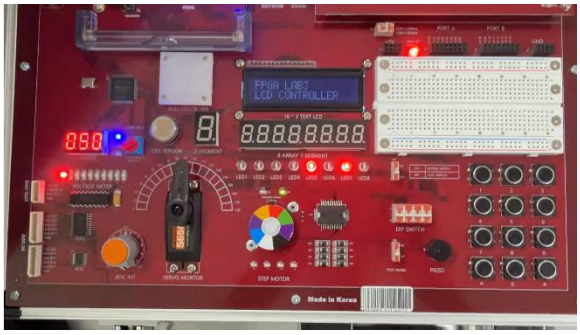
Z의 아스키 코드에 해당하는 LED 점등



줄바꿈 입력



줄바꿈의 아스키 코드에 해당하는 LED



9. 종합 고찰 및 결론

종합 고찰

본 실험에서는 LED PWM, RGB PWM, 피에조, 스텝 모터, MMSS 시계, 문자 LCD, UART Echo를 설계하고 시뮬레이션과 fpga 보드 구현을 통해 이전 실험들에 비해 보다 복잡한 응용 회로의 동작을 검토하였다. 지난 실험에서 설계했던 조합논리회로 및 순차논리회로와 달리 이번 실험의 응용 회로에서는 피에조, LCD 패널, 스텝 모터, Full color led 등 다양한 출력을 활용하였다. 대부분의 실험에서는 순차논리회로에서 학습했던 상태의 개념이 탑재되어 있었다.

이번 실험에서는 시뮬레이션의 파형으로 확인하는 것보다 실제 보드 위에 구현하여 확인하는 것이 훨씬 더 직관적이었다. 특히, UART Echo나 문자 LCD 실험의 경우, 보드에서 LED나 LCD 패널로 확인하는 것이 훨씬 직관적이었다.

결론

본 실험을 통해 여러 FPGA 응용회로를 RTL로 설계하고, 시뮬레이션, 합성, 구현, 비트스트림 생성 및 FPGA 보드 위 구현으로 이어지는 설계 과정을 수행하였다. 특히 실험7 UART Echo의 시연을 통해 FPGA 보드와 외부 기기의 상호 작용에 대해서 경험할 수 있었다.

10. 참고 문헌

[1] Microchip Technology Inc., "Beginner Lab 7: Universal Asynchronous Receiver Transmitter (UART) Polling Mode," Developer Help, "Overview."